

Portfolio Management and Risk Consultation

FINSTAD Case 2 – Group 3 (Charlie)

CRUZ, Ricardo Miguel Iñigo GALEDO, Enrique Lorenzo Hermoso
SEBALLOS, Josiah Dweyn Panganiban SEECHUNG, Camille Castro
SISON, Aaron Joshua Estacio

2026-08-17

Table of contents

1	Setup and Library Loading	2
2	Data Ingestion and Preparation	2
2.1	Daily Log Return Computation	3
2.2	Data Validation and Summary Check	4
3	Exploratory Financial Analysis	4
3.1	Quantitative Design System (QVRS Theme & Colors)	5
3.2	Historical Price & Return Trajectories	6
3.3	Distributional Shape: Histograms and Empirical Density vs Normal Reference	8
3.4	Quantile-Quantile (Q-Q) Tail Diagnostics	9
3.5	Return Dispersion & Extreme Outlier Comparison	10
3.6	Cumulative Performance Comparison (Growth of \$1.00)	11
3.7	Correlation Structure	12
4	Portfolio Construction (Equal-Weight vs Monthly Rebalanced)	13
4.1	Comparative Performance Evaluation	15
4.2	Cumulative Portfolio Growth Comparison	15
4.3	Portfolio Asset Weight Drift Analysis	16
5	Portfolio Optimization (Efficient Frontier, GMV, Max Sharpe)	18
6	Portfolio Risk Assessment (VaR, CVaR, Monte Carlo, Stress Testing)	18
7	Executive Investment Recommendation	18

1 Setup and Library Loading

Load the quantitative finance, portfolio analytics, and data visualization libraries required for the analysis.

```
# Core data manipulation and visualization
library(tidyverse)
library(lubridate)
library(scales)

# Time series and quantitative financial analysis
library(xts)
library(zoo)
library(PerformanceAnalytics)
library(PortfolioAnalytics)

# Solvers for portfolio optimization
library(ROI)
library(ROI.plugin.quadprog)

# Table formatting and visual aesthetics
library(corrplot)
library(knitr)
library(kableExtra)
```

2 Data Ingestion and Preparation

Load the compiled master dataset containing historical daily close prices for Group 3's asset universe (TEL, MER, AEV, NVDA, META, BTC, SPY) and the risk-free rate benchmark over the sample period (January 2, 2020 – December 31, 2025).

```
# Define path to master dataset
data_path <- file.path("../", "data", "master_dataset.csv")

if (!file.exists(data_path)) {
  data_path <- file.path("data", "master_dataset.csv")
}

raw_data <- read_csv(data_path, show_col_types = FALSE)

# Preview structure
glimpse(raw_data)
```

```
Rows: 1,509
Columns: 9
```

```

$ Date      <date> 2020-01-02, 2020-01-03, 2020-01-06, 2020-01-07, 2020-01-08,
  ↪ 2~
$ TEL       <dbl> 12.94961, 13.00212, 12.92992, 13.35655, 13.46156, 13.75691,
  ↪ 13~
$ MER       <dbl> 8.605905, 8.605905, 8.605905, 8.605905, 8.605905, 8.605905,
  ↪ 8.~
$ AEV       <dbl> 8.803739, 8.803739, 8.803739, 8.803739, 8.803739, 8.803739,
  ↪ 8.~
$ NVDA      <dbl> 5.963803, 5.868346, 5.892957, 5.964300, 5.975487, 6.041114,
  ↪ 6.~
$ META      <dbl> 207.9538, 206.8535, 210.7493, 211.2053, 213.3465, 216.3997,
  ↪ 21~
$ BTC       <dbl> 6985.470, 7344.884, 7769.219, 8163.692, 8079.863, 7879.071,
  ↪ 81~
$ SPY       <dbl> 296.1252, 293.8830, 295.0041, 294.1746, 295.7424, 297.7478,
  ↪ 29~
$ RF_Rate   <dbl> 0.0525, 0.0525, 0.0525, 0.0525, 0.0525, 0.0525, 0.0525,
  ↪ 0.0525~

```

2.1 Daily Log Return Computation

Compute daily log returns for all portfolio assets and the benchmark:

$$r_{i,t} = \ln \left(\frac{P_{i,t}}{P_{i,t-1}} \right)$$

```

# Sort chronologically by date
prices_df <- raw_data %>%
  mutate(Date = as.Date(Date)) %>%
  arrange(Date)

# Extract asset price columns
asset_cols <- c("TEL", "MER", "AEV", "NVDA", "META", "BTC", "SPY")

# Convert matrix to xts time series object
prices_mat <- as.matrix(prices_df[, asset_cols])
prices_xts <- xts(prices_mat, order.by = prices_df[["Date"]])

# Compute daily log returns
returns_xts <- Return.calculate(prices_xts, method = "log")[-1, ]

# Separate portfolio assets from benchmark
portfolio_assets <- c("TEL", "MER", "AEV", "NVDA", "META", "BTC")
benchmark_asset <- "SPY"

```

```

asset_returns_xts <- returns_xts[, portfolio_assets]
benchmark_returns_xts <- returns_xts[, benchmark_asset]

# Risk-free rate (convert annual T-Bill rate to daily log-equivalent rate)
rf_annual <- mean(prices_df[["RF_Rate"]], na.rm = TRUE)
rf_daily <- log(1 + rf_annual) / 252

cat("Dataset span:", format(min(index(returns_xts))), "to",
    ↪ format(max(index(returns_xts))), "\n")

```

Dataset span: 2020-01-03 to 2026-01-02

```
cat("Total trading days:", nrow(returns_xts), "\n")
```

Total trading days: 1508

```
cat("Annualized Risk-Free Rate:", percent(rf_annual, accuracy = 0.01), "\n")
```

Annualized Risk-Free Rate: 5.25%

```
cat("Daily Log Risk-Free Rate:", percent(rf_daily, accuracy = 0.0001), "\n")
```

Daily Log Risk-Free Rate: 0.0203%

2.2 Data Validation and Summary Check

Verify data integrity, check for missing values, and print head records of returns.

```

# Verify no missing values in returns time series
missing_summary <- colSums(is.na(returns_xts))
stopifnot(all(missing_summary == 0))

# Format first 6 rows for table display
returns_head_df <- as.data.frame(head(returns_xts)) %>%
  rownames_to_column(var = "Date") %>%
  mutate(across(-Date, ~ sprintf("%.4f", .x)))

kbl(returns_head_df, booktabs = TRUE, align = "c") %>%
  kable_styling(latex_options = c("striped", "hold_position"),
    ↪ bootstrap_options = c("striped", "hover"))

```

3 Exploratory Financial Analysis

```

# Compute descriptive statistics
stats_mat <- table.Stats(asset_returns_xts)

```

Table 1: First 6 Days of Asset Daily Log Returns (2020)

Date	TEL	MER	AEV	NVDA	META	BTC	SPY
2020-01-03	0.0040	0.0000	0.0000	-0.0161	-0.0053	0.0502	-0.0076
2020-01-06	-0.0056	0.0000	0.0000	0.0042	0.0187	0.0562	0.0038
2020-01-07	0.0325	0.0000	0.0000	0.0120	0.0022	0.0495	-0.0028
2020-01-08	0.0078	0.0000	0.0000	0.0019	0.0101	-0.0103	0.0053
2020-01-09	0.0217	0.0000	0.0000	0.0109	0.0142	-0.0252	0.0068
2020-01-10	0.0029	0.0000	0.0000	0.0053	-0.0011	0.0358	-0.0029

```
# Convert to data frame for clean rendering
stats_df <- as.data.frame(stats_mat) %>%
  rownames_to_column(var = "Metric")

kbl(stats_df, booktabs = TRUE, digits = 4) %>%
  kable_styling(latex_options = c("striped", "hold_position", "scale_down"),
    ↪ bootstrap_options = c("striped", "hover"))
```

3.1 Quantitative Design System (QVRS Theme & Colors)

```
# QVRS Economist Color Tokens
qvrs_colors <- c(
  "TEL" = "#1F2E7A", # Chicago 30
  "MER" = "#2E45B8", # Chicago 45
  "AEV" = "#475ED1", # Chicago 55
  "NVDA" = "#1DC9A4", # Hong Kong 45
  "META" = "#F97A1F", # Singapore 55
  "BTC" = "#E3120B", # Economist Red
  "SPY" = "#595959" # London 35
)

# Custom Economist/QVRS ggplot2 theme
theme_quant <- function() {
  theme_minimal() +
  theme(
    text = element_text(color = "#0D0D0D"),
    plot.title = element_text(face = "bold", size = 12, margin = margin(b
    ↪ = 4)),
    plot.subtitle = element_text(size = 9.5, color = "#333333", margin =
    ↪ margin(b = 8)),
    plot.caption = element_text(size = 8, color = "#595959", hjust = 0,
    ↪ margin = margin(t = 6)),
```

Table 2: Descriptive Statistics for Portfolio Assets (2020–2025)

Metric	TEL	MER	AEV	NVDA	META	BTC
Observations	1508.0000	1508.0000	1508.0000	1508.0000	1508.0000	1508.0000
NAs	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Minimum	-0.2703	-0.2116	-0.4724	-0.2040	-0.3064	-0.4647
Quartile 1	-0.0098	0.0000	0.0000	-0.0157	-0.0117	-0.0160
Median	0.0000	0.0000	0.0000	0.0032	0.0010	0.0011
Arithmetic Mean	0.0003	0.0005	-0.0005	0.0023	0.0008	0.0017
Geometric Mean	0.0001	0.0003	-0.0010	0.0017	0.0004	0.0009
Quartile 3	0.0101	0.0000	0.0000	0.0215	0.0141	0.0203
Maximum	0.1548	0.2868	0.3065	0.2181	0.2093	0.1915
SE Mean	0.0005	0.0005	0.0007	0.0009	0.0007	0.0010
LCL Mean (0.95)	-0.0007	-0.0005	-0.0019	0.0006	-0.0007	-0.0003
UCL Mean (0.95)	0.0013	0.0014	0.0009	0.0040	0.0022	0.0037
Variance	0.0004	0.0004	0.0008	0.0011	0.0008	0.0015
Stdev	0.0199	0.0193	0.0279	0.0333	0.0279	0.0393
Skewness	-1.7687	2.0662	-3.3581	0.0434	-1.0731	-1.2226
Kurtosis	31.4844	83.2401	109.4985	4.0565	22.0943	16.2966

```

axis.title = element_text(size = 9, face = "bold", color = "#1A1A1A"),
axis.text = element_text(size = 8, color = "#333333"),
panel.grid.minor = element_blank(),
panel.grid.major = element_line(color = "#F2F2F2", linewidth = 0.3),
legend.position = "bottom",
legend.title = element_blank(),
axis.ticks = element_blank(),
strip.background = element_rect(fill = "#EBEDFA", color = NA),
strip.text = element_text(face = "bold", size = 9, color = "#141F52")
)
}

```

3.2 Historical Price & Return Trajectories

```

# Reshape returns for tidy plotting
returns_tidy <- as.data.frame(returns_xts) %>%
  rownames_to_column(var = "Date") %>%
  mutate(Date = as.Date(Date)) %>%
  pivot_longer(-Date, names_to = "Asset", values_to = "Return") %>%
  mutate(Asset = factor(Asset, levels = names(qvrs_colors)))

```

```

ggplot(returns_tidy, aes(x = Date, y = Return, color = Asset)) +
  geom_line(linewidth = 0.35, alpha = 0.85) +
  facet_wrap(~ Asset, ncol = 2, scales = "free_y") +
  scale_color_manual(values = qvrs_colors) +
  scale_y_continuous(labels = label_percent(accuracy = 1)) +
  scale_x_date(date_labels = "%Y", date_breaks = "2 years") +
  labs(
    title = "Daily Log Return Dynamics and Volatility Clustering",
    subtitle = "Sample Period: January 2, 2020 through December 31, 2025",
    x = NULL,
    y = "Daily Log Return"
  ) +
  theme_quant() +
  theme(legend.position = "none")

```



Figure 1: Daily Log Return Trajectories Across Portfolio Universe (2020–2025)

3.3 Distributional Shape: Histograms and Empirical Density vs Normal Reference

```
# Compute normal distribution parameters per asset for reference curves
norm_params <- returns_tidy %>%
  group_by(Asset) %>%
  summarise(
    mean = mean(Return, na.rm = TRUE),
    sd = sd(Return, na.rm = TRUE),
    .groups = "drop"
  )

# Grid of x values for theoretical normal density overlay
grid_df <- norm_params %>%
  group_by(Asset) %>%
  reframe(
    x = seq(mean - 4 * sd, mean + 4 * sd, length.out = 200),
    y = dnorm(x, mean = mean, sd = sd)
  )

ggplot(returns_tidy, aes(x = Return)) +
  geom_histogram(
    aes(y = after_stat(density)),
    bins = 60,
    fill = "#D6DBF5",
    color = "#1F2E7A",
    linewidth = 0.2,
    alpha = 0.7
  ) +
  geom_density(aes(color = "Empirical Density"), linewidth = 0.8) +
  geom_line(
    data = grid_df,
    aes(x = x, y = y, color = "Gaussian Normal"),
    linetype = "dashed",
    linewidth = 0.75
  ) +
  facet_wrap(~ Asset, scales = "free", ncol = 3) +
  scale_color_manual(
    values = c("Empirical Density" = "#141F52", "Gaussian Normal" =
      ↪ "#E3120B")
  ) +
  scale_x_continuous(labels = label_percent(accuracy = 1)) +
  labs(
    title = "Empirical Distribution Shape: Heavy Tails and Non-Normality",
    subtitle = "Comparison of empirical kernel density against theoretical
      ↪ normal distribution",
```



```
x = "Daily Log Return",
y = "Density"
) +
theme_quant()
```

Empirical Distribution Shape: Heavy Tails and Non-Normality

Comparison of empirical kernel density against theoretical normal distribution

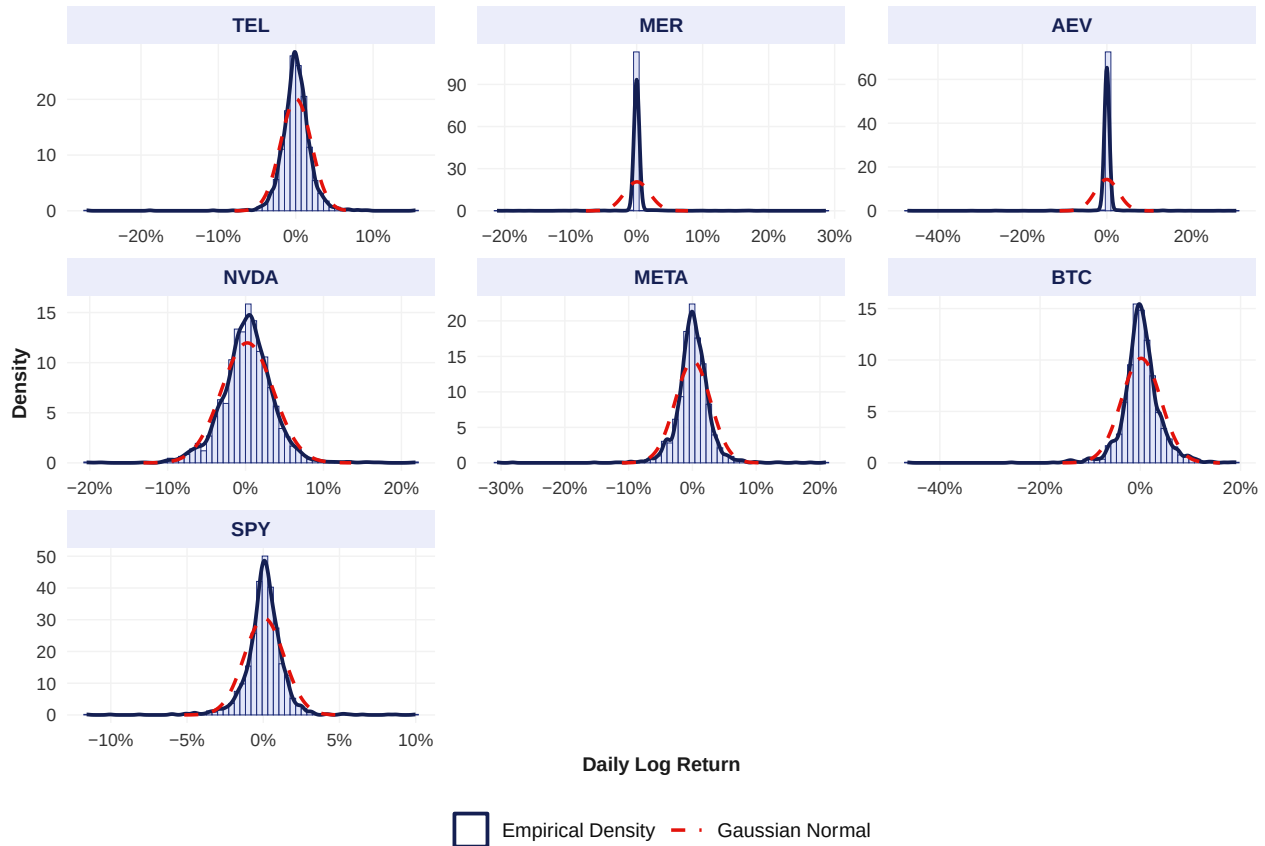


Figure 2: Empirical Return Distributions vs Gaussian Reference Curves

3.4 Quantile-Quantile (Q-Q) Tail Diagnostics

```
ggplot(returns_tidy, aes(sample = Return)) +
  stat_qq_line(color = "#E3120B", linewidth = 0.7, linetype = "dashed") +
  stat_qq(color = "#1F2E7A", alpha = 0.5, size = 0.8) +
  facet_wrap(~ Asset, scales = "free", ncol = 3) +
  labs(
    title = "Quantile-Quantile Tail Departure from Normality",
    subtitle = "S-shaped curves confirm pronounced fat-tail behavior across
    ↪ asset returns",
    x = "Theoretical Gaussian Quantiles",
    y = "Sample Quantiles"
```

```
) +  
theme_quant()
```

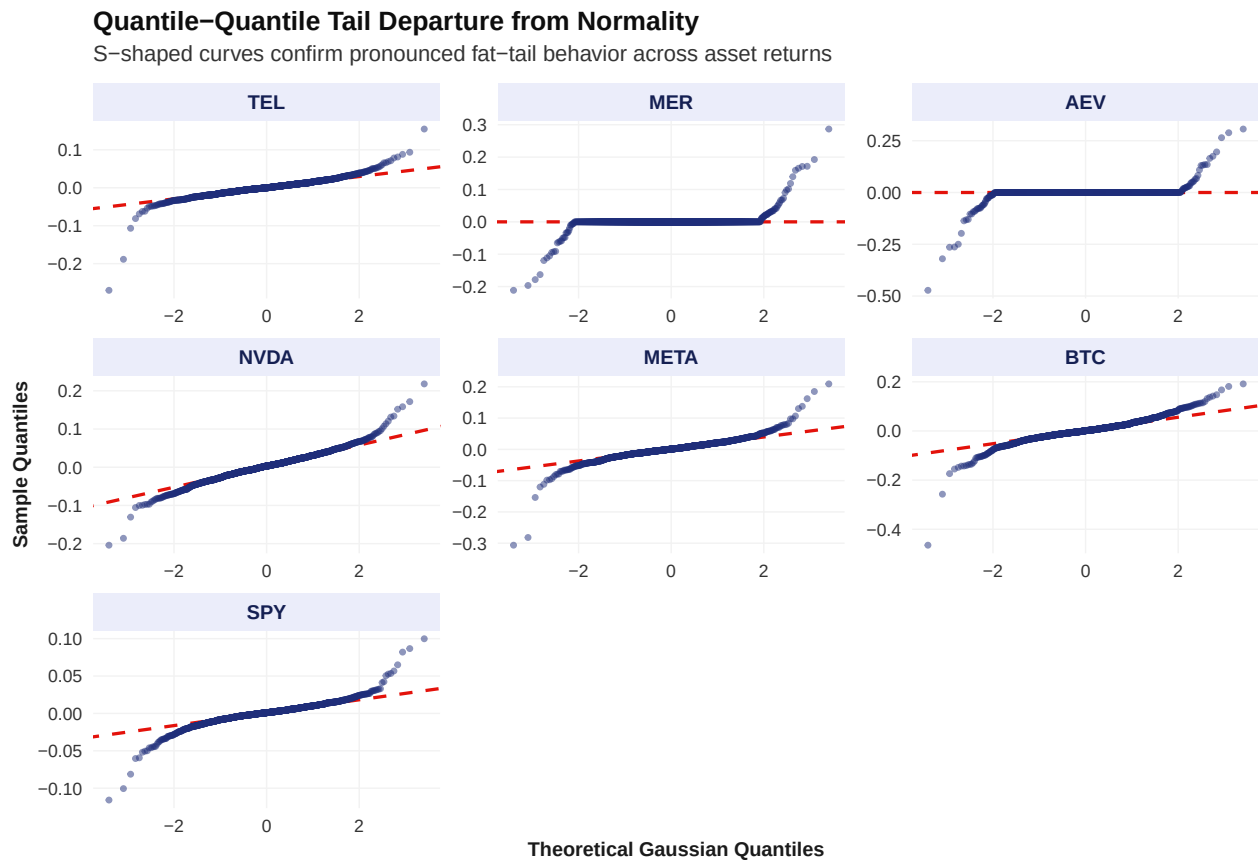


Figure 3: Normal Q-Q Diagnostics Demonstrating Heavy Tail Leptokurtosis

3.5 Return Dispersion & Extreme Outlier Comparison

```
ggplot(returns_tidy, aes(x = Asset, y = Return, fill = Asset)) +  
  geom_boxplot(outlier.color = "#E3120B", outlier.size = 1, outlier.alpha =  
    ↪ 0.5, linewidth = 0.4) +  
  scale_fill_manual(values = qvrs_colors) +  
  scale_y_continuous(labels = label_percent(accuracy = 1)) +  
  labs(  
    title = "Cross-Asset Return Dispersion and Extreme Tail Outliers",  
    subtitle = "Comparison of median, interquartile range (IQR), and extreme  
      ↪ shocks",  
    x = "Asset",  
    y = "Daily Log Return"  
  ) +  
  theme_quant() +
```

```
theme(legend.position = "none")
```

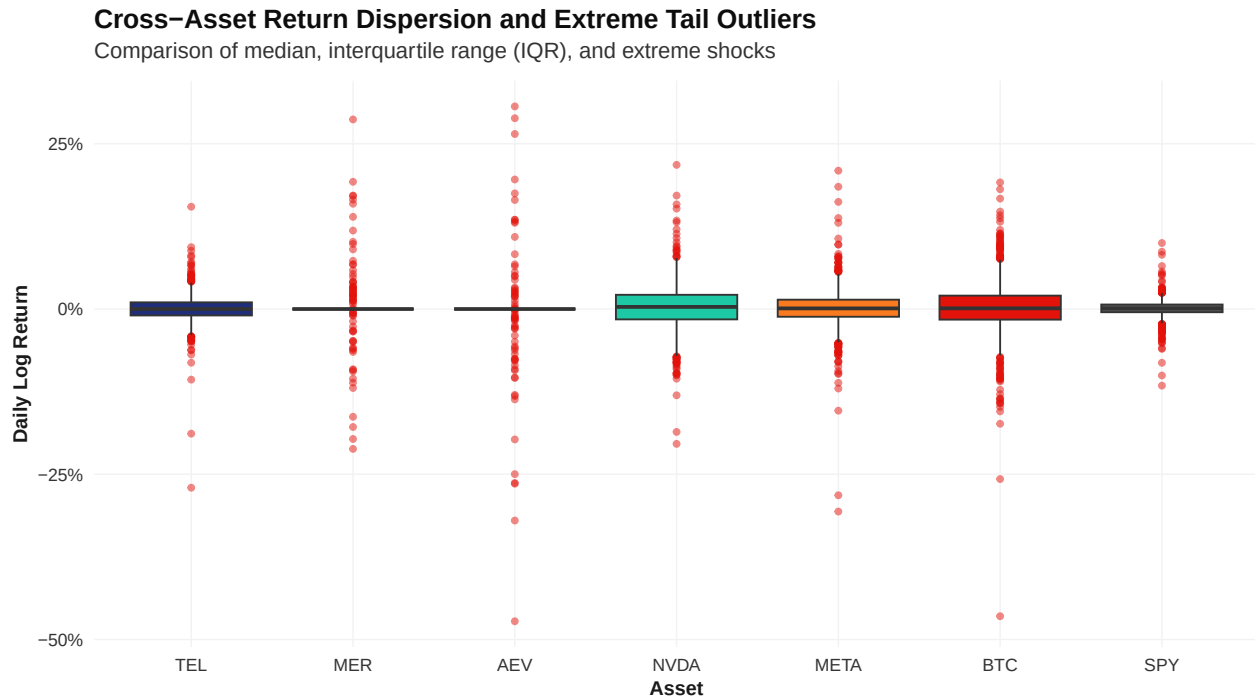


Figure 4: Return Dispersion and Extreme Outlier Boxplots

3.6 Cumulative Performance Comparison (Growth of \$1.00)

```
# Compute cumulative wealth index
cum_returns_df <- as.data.frame(cumprod(1 + returns_xts)) %>%
  rownames_to_column(var = "Date") %>%
  mutate(Date = as.Date(Date)) %>%
  pivot_longer(-Date, names_to = "Asset", values_to = "Wealth") %>%
  mutate(Asset = factor(Asset, levels = names(qvrs_colors)))

ggplot(cum_returns_df, aes(x = Date, y = Wealth, color = Asset)) +
  geom_line(linewidth = 0.75) +
  scale_color_manual(values = qvrs_colors) +
  scale_y_log10(labels = label_dollar(prefix = "USD ")) +
  scale_x_date(date_labels = "%Y", date_breaks = "1 year") +
  labs(
    title = "Cumulative Wealth Index Growth Trajectories (Log Scale)",
    subtitle = "Hypothetical USD 1.00 investment performance from January
  ↪ 2020 to December 2025",
    x = NULL,
    y = "Wealth Index (USD)"
  ) +
```

```
theme_quant()
```

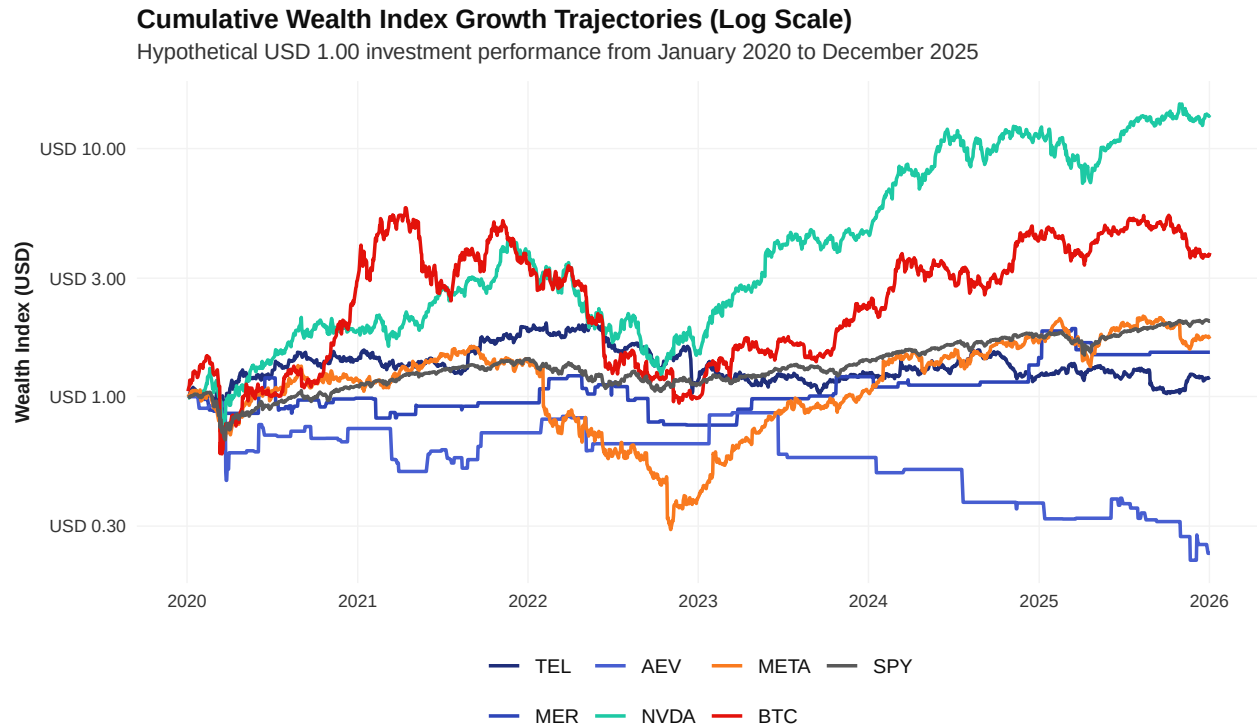


Figure 5: Cumulative Growth Trajectories of USD 1.00 Investment (2020–2025)

3.7 Correlation Structure

```
# Compute pairwise correlation matrix
cor_matrix <- cor(asset_returns_xts, use = "pairwise.complete.obs")

# Render correlation heatmap
corrplot(
  cor_matrix,
  method = "color",
  type = "upper",
  addCoef.col = "black",
  tl.col = "black",
  number.cex = 0.8,
  mar = c(0, 0, 1, 0)
)
```

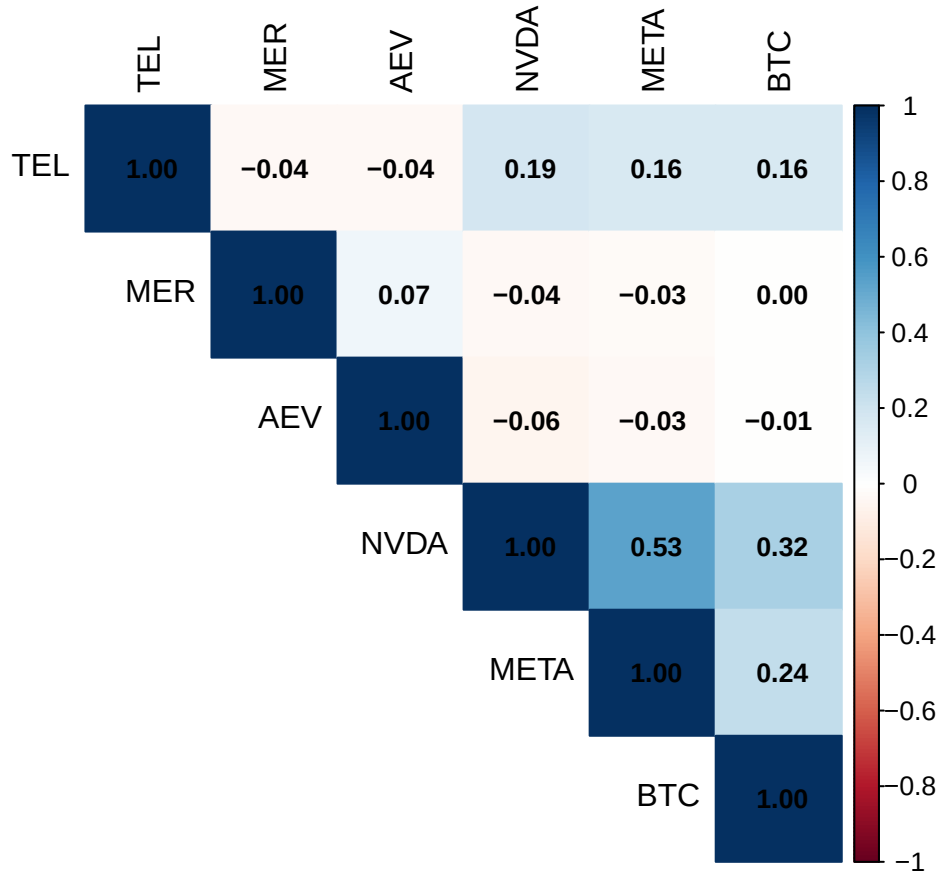


Figure 6: Asset Return Correlation Heatmap (Group 3 Universe)

4 Portfolio Construction (Equal-Weight vs Monthly Rebalanced)

This section evaluates the performance of equal-weighted asset allocations under two distinct execution regimes: a static Buy-and-Hold strategy and a disciplined Monthly Rebalanced strategy. The 6-asset portfolio universe (TEL, MER, AEV, NVDA, META, BTC) starts with equal baseline weights ($w_i = 1/6 \approx 16.67\%$) on January 2, 2020 and is benchmarked against the SPDR S&P 500 ETF Trust (SPY).

```
# Equal weight allocation vector across the 6 portfolio assets
w_eq <- rep(1 / 6, 6)
names(w_eq) <- colnames(asset_returns_xts)

# Strategy 1: Buy-and-Hold Equal-Weighted Portfolio
port_bnh <- Return.portfolio(
  R = asset_returns_xts,
  weights = w_eq,
  verbose = TRUE
)
```

```

r_bnh <- port_bnh$returns
weights_bnh <- port_bnh$EOP.Weight

# Strategy 2: Monthly Rebalanced Equal-Weighted Portfolio
port_reb <- Return.portfolio(
  R = asset_returns_xts,
  weights = w_eq,
  rebalance_on = "months",
  verbose = TRUE
)
r_reb <- port_reb$returns
weights_reb <- port_reb$EOP.Weight

# Risk-free rate metrics (5.25% per annum)
rf_annual <- 0.0525
rf_daily <- log(1 + rf_annual) / 252

# Performance evaluation helper function
eval_portfolio <- function(r_series, name) {
  cum_ret <- exp(sum(r_series)) - 1
  ann_ret <- as.numeric(Return.annualized(r_series, geometric = TRUE))
  ann_sd <- as.numeric(StdDev.annualized(r_series))
  sharpe <- (ann_ret - rf_annual) / ann_sd
  max_dd <- as.numeric(maxDrawdown(r_series))

  data.frame(
    Strategy = name,
    `Cumulative Return` = sprintf("%.2f%%", cum_ret * 100),
    `CAGR (Annualized)` = sprintf("%.2f%%", ann_ret * 100),
    `Annual Volatility` = sprintf("%.2f%%", ann_sd * 100),
    `Sharpe Ratio (Rf=5.25%)` = sprintf("%.4f", sharpe),
    `Max Drawdown` = sprintf("%.2f%%", max_dd * 100),
    check.names = FALSE
  )
}

perf_summary_df <- rbind(
  eval_portfolio(r_bnh, "Equal-Weight (Buy & Hold)"),
  eval_portfolio(r_reb, "Equal-Weight (Monthly Rebalanced)"),
  eval_portfolio(returns_xts[, "SPY"], "SPDR S&P 500 ETF (SPY Benchmark)")
)

```

4.1 Comparative Performance Evaluation

```
safe_kable(perf_summary_df, caption = "Portfolio Performance Summary
↪ Comparison (2020--2025)")
```

Table 3: Portfolio Performance Summary Comparison (2020–2025)

Strategy	Cumulative Return	CAGR (Annualized)	Annual Volatility	Sharpe Ratio (Rf=5.25%)	Max
Equal-Weight (Buy & Hold)	401.41%	24.35%	31.96%	0.5977	61.56
Equal-Weight (Monthly Rebalanced)	227.20%	18.66%	23.16%	0.5790	49.21
SPDR S&P 500 ETF (SPY Benchmark)	129.48%	12.42%	20.79%	0.3448	35.75

4.2 Cumulative Portfolio Growth Comparison

```
# Merge portfolio return series
port_returns_combined <- cbind(r_bnh, r_reb, returns_xts[, "SPY"])
colnames(port_returns_combined) <- c("Buy_Hold", "Monthly_Rebalanced",
↪ "SPY_Benchmark")

# Compute cumulative wealth index
port_growth_df <- as.data.frame(cumprod(1 + port_returns_combined)) %>%
  rownames_to_column(var = "Date") %>%
  mutate(Date = as.Date(Date)) %>%
  pivot_longer(-Date, names_to = "Strategy", values_to = "Wealth") %>%
  mutate(
    Strategy = case_when(
      Strategy == "Buy_Hold" ~ "Equal-Weight (Buy & Hold)",
      Strategy == "Monthly_Rebalanced" ~ "Equal-Weight (Monthly
↪ Rebalanced)",
      Strategy == "SPY_Benchmark" ~ "SPDR S&P 500 ETF (SPY)"
    ),
    Strategy = factor(
      Strategy,
      levels = c("Equal-Weight (Buy & Hold)", "Equal-Weight (Monthly
↪ Rebalanced)", "SPDR S&P 500 ETF (SPY)")
    )
  )

strat_colors <- c(
  "Equal-Weight (Buy & Hold)" = "#E3120B",          # Economist Red
  "Equal-Weight (Monthly Rebalanced)" = "#1F2E7A", # Chicago 30
  "SPDR S&P 500 ETF (SPY)" = "#595959"            # London 35
)
```

```
ggplot(port_growth_df, aes(x = Date, y = Wealth, color = Strategy, linetype
  ↳ = Strategy)) +
  geom_line(linewidth = 0.85) +
  scale_color_manual(values = strat_colors) +
  scale_linetype_manual(values = c("solid", "solid", "dashed")) +
  scale_y_log10(labels = label_dollar(prefix = "USD ")) +
  scale_x_date(date_labels = "%Y", date_breaks = "1 year") +
  labs(
    title = "Cumulative Wealth Growth Comparison (Log Scale)",
    subtitle = "Performance of USD 1.00 allocation from January 2020 through
      ↳ December 2025",
    x = NULL,
    y = "Wealth Index (USD)"
  ) +
  theme_quant()
```

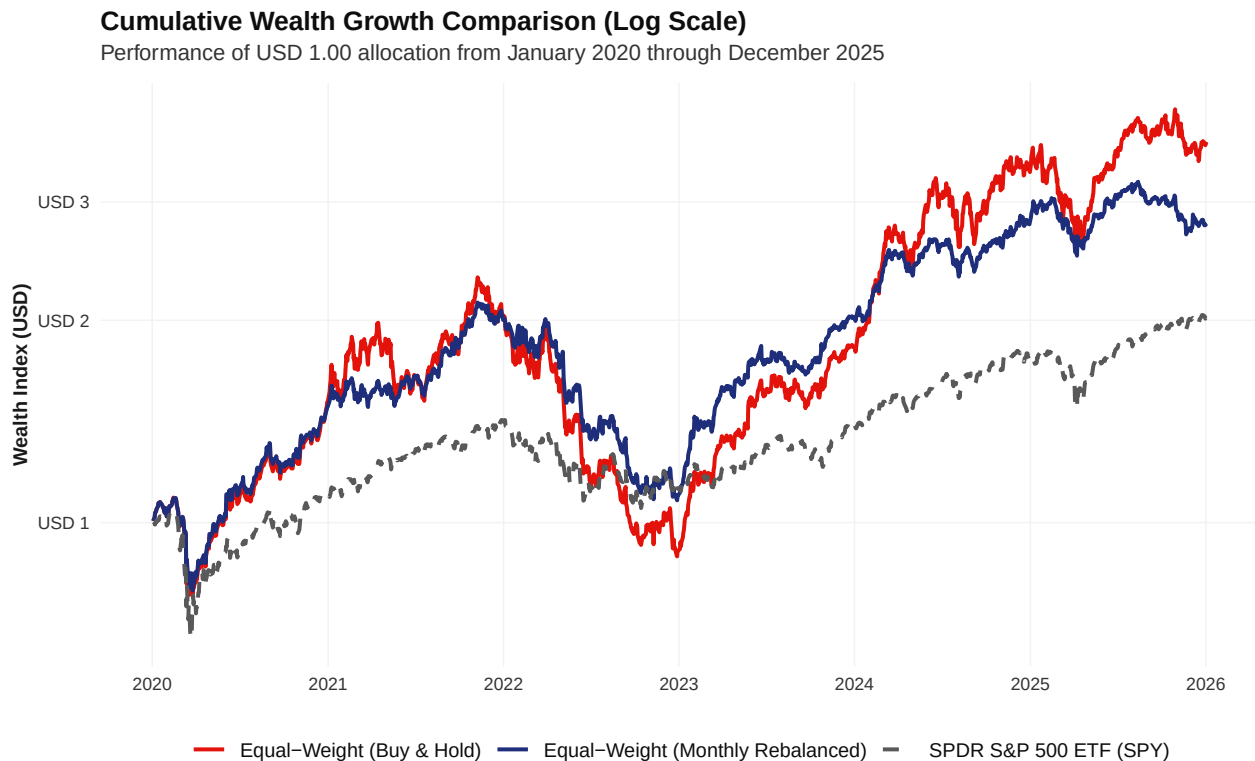


Figure 7: Cumulative Growth Trajectories of USD 1.00 Investment Across Strategies

4.3 Portfolio Asset Weight Drift Analysis

```
# Reshape end-of-period weights for Buy & Hold strategy
weights_bnh_df <- as.data.frame(weights_bnh) %>%
  rownames_to_column(var = "Date") %>%
```



```

mutate(Date = as.Date(Date)) %>%
pivot_longer(-Date, names_to = "Asset", values_to = "Weight") %>%
mutate(Asset = factor(Asset, levels = names(qvrs_colors)))

ggplot(weights_bnh_df, aes(x = Date, y = Weight, fill = Asset)) +
  geom_area(alpha = 0.85, color = "#FFFFFF", linewidth = 0.1) +
  scale_fill_manual(values = qvrs_colors) +
  scale_y_continuous(labels = label_percent(accuracy = 1)) +
  scale_x_date(date_labels = "%Y", date_breaks = "1 year") +
  labs(
    title = "Asset Allocation Drift in Buy-and-Hold Execution",
    subtitle = "Evolution of asset weights driven by differential market
      ↪ performance (2020--2025)",
    x = NULL,
    y = "Portfolio Weight Share"
  ) +
  theme_quant()

```

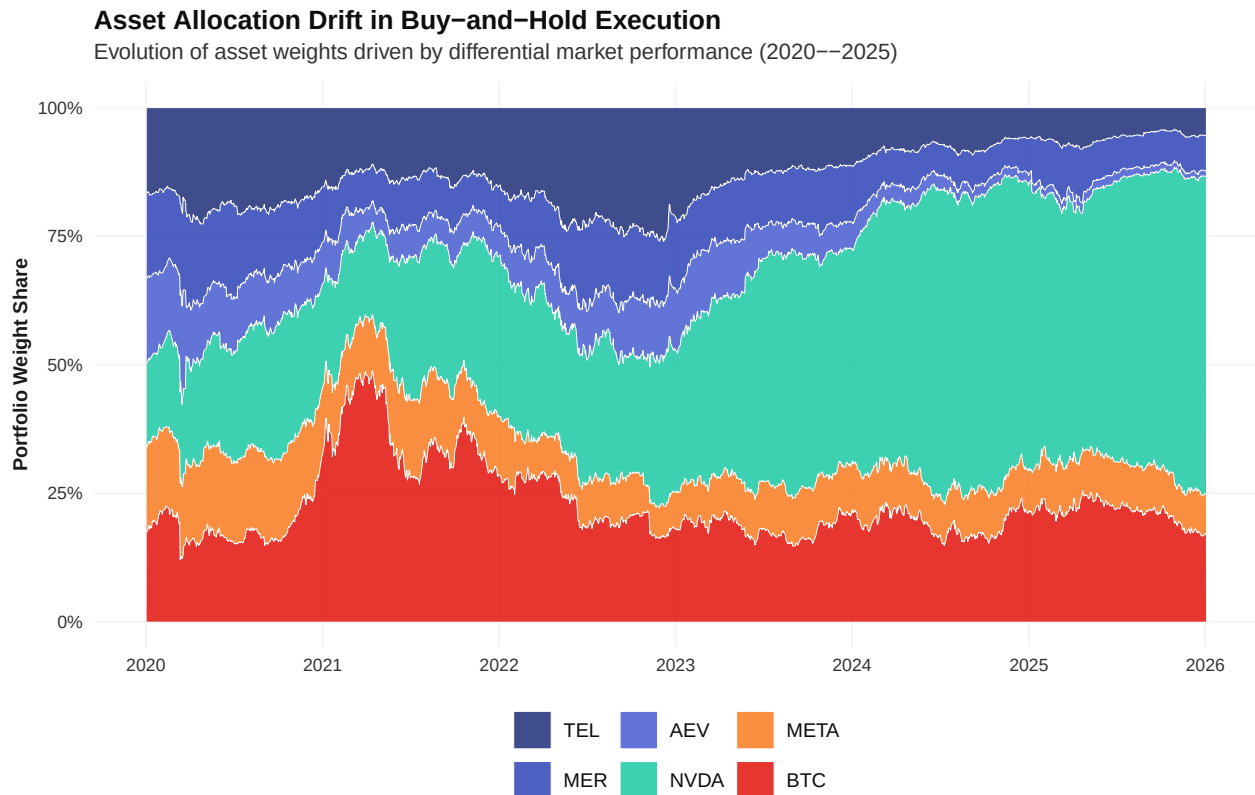


Figure 8: Asset Weight Drift Over Time in the Un-Rebalanced Buy-and-Hold Portfolio

5 Portfolio Optimization (Efficient Frontier, GMV, Max Sharpe)

```
# Placeholder for Portfolio Optimization routines
```

6 Portfolio Risk Assessment (VaR, CVaR, Monte Carlo, Stress Testing)

```
# Placeholder for Portfolio Risk Assessment routines
```

7 Executive Investment Recommendation

```
# Placeholder for final quantitative recommendation summary
```